SVM Cardiac Arrhythmia Classifier — Start-Up & Model-Programming Guide

GF180MCU / wafer.space shuttle · slot 0p5x1

Adam Handwerger · Portland State University

2026

1. Device overview

This chip is a **5-class RBF-kernel Support-Vector-Machine (SVM) cardiac-arrhythmia classifier**. It ingests heartbeat feature vectors, evaluates the RBF kernel against a stored set of support vectors (SVs), applies the trained coefficients, and reports the predicted class (Normal, PVC, AFib, VT, SVT).

Attribute	Value
Technology	GF180MCU 180 nm, 5 V flow (`gf180mcu_fd_sc_mcu7t5v0`, `gf180mcu_fd_io`)
Die / core	1936 × 5122 µm die · 1052 × 4238 µm core (slot `0p5x1`)
Clock	25 MHz (40 ns period) on the dedicated `clk` pad — see note ‡
Number format	**Q6.10** fixed-point (16-bit; 1024 = 1.000)
Support vectors	**600** (120 per class)
Feature dimension	256
Classes	5
Host interface	**SPI slave** (mode 0, MSB-first)
Verified accuracy	**98.67 % (296/300)** — full-dataset RTL cosim on the PhysioNet test set

‡ **Clock note:** timing closes at 25 MHz at the typical (`tt_025C_5v00`, +14 ns) and fast corners with large margin. At the worst-case slow corner (`ss_125C_4v50` — slow silicon, 125 °C, 4.5 V) setup is –5.07 ns; for guaranteed operation at that extreme corner, derate to ~20 MHz. DRC/LVS/antenna are clean and hold passes at all corners. For a cardiac classifier (~1–3 beats/s) either speed is far more than sufficient.

The chip holds three kinds of data. Two are **field-programmable** (loaded after every power-up); one is **fixed in silicon**:

Data	Storage	Programmable?
Alpha (dual) coefficients, 600 × 16	on-chip `alpha_sram` (4× SRAM macros)	**yes** — via SPI
Support-vector / input matrix	off-chip RAM (host-served) + on-chip `feature_sram`	**yes** — via host RAM

Data	Storage	Programmable?
exp() kernel LUT (8 × 16)	combinational logic (baked in)	no — model-independent

Because the model lives in SRAM/RAM, the **exact same silicon can run a re-trained model** (see §6). Only the exponential lookup is hard-wired, and it never needs to change.

2. Pin map

Dedicated pads: `clk`, `rst_n`. All other I/O is on the input-only and bidirectional pads.

Function	Pad(s)	Dir
SPI clock `sclk`	`input[0]`	in
SPI chip-select `cs_n` (active low)	`input[1]`	in
SPI data-in `mosi`	`input[2]`	in
SPI data-out `miso`	`bidir[36]`	out
Off-chip RAM address	`bidir[18:0]`	out
Off-chip RAM read-enable `ram_ren`	`bidir[19]`	out
Off-chip RAM read-data	`bidir[35:20]`	in
`done`	`bidir[37]`	out
`error`	`bidir[38]`	out
`sample_rdy`	`bidir[39]`	out
`class_out[2:0]`	`bidir[42:40]`	out
`kernel_valid`	`bidir[43]`	out
Core power	`vdd_pads` / `vss_pads`	—
I/O power	`dvdd_pads` / `dvss_pads`	—

3. Power-up sequence

1. **Apply ground** (`vss`, `dvss`) first.
2. **Apply the 5 V rails** — core (`vdd`) and I/O (`dvdd`) — to **5.0 V nominal** (4.5–5.5 V). The core and I/O share the 5 V domain in this build.
3. **Start the clock**: 25 MHz on `clk` (40 ns period). It may be applied before or after the rails are stable, but hold reset until it is running.
4. **Assert reset**: drive `rst_n` **low** for at least 8 clock cycles, then release high. On reset the classifier returns to IDLE, clears sticky errors, and loads the default `gamma = 0x0100` (0.25).
5. The device is now idle and ready to be programmed over SPI.

   The alpha SRAM contents are **volatile** — they are undefined after power-up and **must be loaded before the first classification** (§5). Keep the model in the host's non-volatile storage and load it as part of every boot.

4. Host (SPI) protocol

- **Mode 0**, **MSB-first**, one byte per transfer, `cs_n` framing.
- Each transaction begins with a **header byte** = `{rd_nwr, addr[6:0]}`: bit 7 = **1 for read, 0 for write**; bits 6:0 = register address.
- A **write** header is followed by that register's payload bytes.
- A **read** header is followed by clocking out the register bytes on `miso` (send dummy `0x00` on `mosi`).

Register map

Reg	Addr	Access	Payload
CTRL	0x00	W	1 byte — bit0 = start
NSAMP	0x01	W	2 bytes — number of heartbeats to process
NSVPC	0x02	W	support-vectors-per-class (5 fields)
PARAM	0x03	W	3 bytes — [param_addr, data_hi, data_lo]
ALPHA	0x04	W	alpha coefficient stream (see §5)
STATUS	0x40	R	2 bytes — status word (below)
GAMMA	0x41	R	2 bytes — current gamma (Q6.10)
C	0x42	R	2 bytes — current C (Q6.10)
KERNEL	0x43	R	last kernel value
SCORES	0x44	R	per-class decision scores

PARAM addresses: 0x00 = gamma, 0x01 = C. Example — set gamma = 0.5 (Q6.10 = 0x0200): write PARAM with [0x00, 0x02, 0x00].

STATUS word (16-bit, MSB first):

Bit(s)	15	14	13	12	11:8	7:3	2:0
Field	sample_rdy	done	error	kernel_valid	error_code	0	class_out

5. Loading a model (first-time programming)

Perform these steps once after each power-up, while the core is IDLE:

1. **Set kernel parameters** — PARAM write gamma (param_addr 0x00) and, if used, C (param_addr 0x01). Default gamma after reset is 0.25 (0x0100); the reference model uses 0.25, so this step can be skipped if unchanged.
2. **Set support-vector counts** — NSVPC write the per-class SV counts (e.g. 120,120,120,120,120 -> 600 total). Total must be **≤ 600**.
3. **Load alpha coefficients** — ALPHA write the NUM_SV × 16-bit dual coefficients, in class-then-index order, MSB-first. These land in the on-chip alpha SRAM.
4. **Stage the SV / input matrix in off-chip RAM** — the host memory (served on the ram_addr/ram_rdata/ram_ren bus) must contain the support-vector matrix and the input heartbeat features, in the layout the core reads (SV block followed by the input beats, 256 features each, Q6.10).
5. **Set the batch size** — NSAMP write the number of heartbeats to classify.

The device is now programmed.

6. Running a classification

1. Ensure the host RAM has the next heartbeat's 256 features available.
2. Write CTRL with start = 1 (bit 0).
3. The core reads features (paced over the RAM bus), evaluates all SVs, accumulates the per-class scores, and takes the arg-max.
4. Poll STATUS (or watch the pins): when **sample_rdy / done** is high, the result is valid. Read **class_out[2:0]** for the predicted class:

class_out	Class
0	Normal
1	PVC (premature ventricular contraction)
2	AFib (atrial fibrillation)
3	VT (ventricular tachycardia)
4	SVT (supraventricular tachycardia)

5. Optionally read `SCORES` (per-class margins) and `KERNEL` for diagnostics.
6. For a batch, the core loops `NSAMP` times, pulsing `sample_rdy` per beat and `done` at the end. If `error` (STATUS bit 13) sets, read `error_code` (bits 11:8); errors are sticky and clear on reset.

7. Changing the model (re-training / field update)

The classifier is **model-agnostic silicon**. To deploy a new model, retrain offline and reload the coefficients — **no silicon change, and the exp() LUT is untouched** (it is the mathematical exponential, independent of the trained model).

Offline (host PC):

1. Train an **RBF-kernel SVM** on the new labelled data (scikit-learn `SVC(kernel='rbf', gamma=0.25, C=1.0)`, one-vs-one, 5 classes). Keep the same **256-feature** vector definition the front-end produces.
2. Extract from the fitted model:
   - `support_vectors_` -> the SV matrix,
   - `dual_coef_` -> the alpha coefficients,
   - `gamma`, and the per-class SV counts (`n_support_`).
3. **Quantize to Q6.10** (multiply by 1024, round, clamp to signed 16-bit). Keep the total SV count **≤ 600**; if training yields more, reduce `C`, prune, or cap SVs per class.
4. Serialize the SV matrix into the host-RAM image layout, and the alphas into the `ALPHA` byte stream (class-then-index order).

On-chip (per power-up):

5. Power up and reset (§3).
6. Program the new model exactly as in §5: `PARAM` (new gamma), `NSVPC` (new counts), `ALPHA` (new coefficients), refresh the off-chip SV matrix, `NSAMP`.
7. Resume classifying (§6).

What you may change without re-spinning silicon: the support vectors, the alpha coefficients, gamma, C, the SV-per-class allocation (≤ 600 total), and the batch size. **What is fixed:** the 256-feature interface, the 5-class output, the Q6.10 number format, and the exp() kernel LUT.

8. Quick reference

- **Reset default gamma:** `0x0100` (0.25, Q6.10). Verify with a `GAMMA` read.
- **Q6.10 conversion:** value × 1024, rounded (e.g. 0.25 -> 256 = `0x0100`; 0.5 -> `0x0200`).
- **Max support vectors:** 600. **Features:** 256. **Classes:** 5.
- **Sanity check after boot:** read `GAMMA` (expect `0x0100`) and `STATUS` (expect idle: `done`/`error` low) to confirm the SPI link before loading a model.